

Internship Project Review Note — Spark/API & Two-Tower Progress

Prepared for mentor review

1. Current status

I am currently working on two parallel internship projects:

- Spark/API project: a production-inspired workflow from PySpark data processing to model output and API serving.
- Two-Tower RetailRocket project: a recommender retrieval project for learning candidate generation, embeddings, and offline top-K evaluation.

Both projects now have runnable artifacts, reports, tests or validation commands, and codebase walkthrough documents. However, I also want to be transparent that I have used Codex/AI assistance extensively to move fast. This helped me produce artifacts quickly, but it also created a risk: some code and configuration were generated faster than I could fully understand them. My current priority is to pause, review the codebase, understand the flow and trade-offs, and align with mentor direction before adding more features.

2. Spark/API project

Goal: simulate a production-style data-to-prediction-serving pipeline:

raw data → EDA → PySpark preprocessing → model training/evaluation → prediction output table → FastAPI lookup API.

Dataset and problem:

- Dataset: Bank Marketing dataset.
- Size: 45,211 rows, 17 columns.
- Target: y = yes/no, whether a customer subscribes to term deposit.
- Positive class yes is about 11.7%, so the dataset is imbalanced.
- duration is excluded because it is only known after the phone call; using it for pre-campaign prediction would cause leakage.

Completed work:

- Pandas EDA with charts and notes.
- PySpark preprocessing using StringIndexer, OneHotEncoder, and VectorAssembler.
- Logistic Regression training/evaluation.
- Model comparison: threshold tuning, weighted Logistic Regression, and GBTCClassifier.
- FastAPI MVP with /health and /prediction/{user_id}.
- API supports 200 response, 404 USER_NOT_FOUND, request_id, trace_id, and timestamp.

- Mock 500k prediction table generated for API lookup/cache/load-test simulation.

Important model results:

Variant	Threshold	Accuracy	Precision	Recall	F1	AUC
LR default	0.50	0.8941	0.7046	0.1863	0.2946	0.7800
LR best F1	0.20	0.8801	0.4949	0.4591	0.4763	0.7800
Weighted LR best F1	0.60	0.8567	0.4163	0.5127	0.4595	0.7798
GBTClassifier	default	0.8932	0.6890	0.1834	0.2897	0.7940

Interpretation:

- LR threshold 0.5 has high accuracy but low recall because of class imbalance and a conservative threshold.
- LR threshold 0.2 gives the best F1 in the current gate and improves recall from 0.1863 to 0.4591 without collapsing precision.
- GBT has slightly higher AUC, but its default threshold does not improve F1/recall yet.
- **The 500k mock API data is not used for model training or evaluation. It is only used to test API serving behavior.**

Current caveats:

- API currently serves mock 500k prediction rows, not the real model prediction output.
- Cache is still MVP-level; hit/miss metrics are not fully measured yet.
- No load test, stress test, or performance report yet.

- Current modeling workflow uses train/test split; for stricter evaluation, a separate validation set should be used for threshold/model selection before final test evaluation.

3. Two-Tower RetailRocket project

Goal: learn ecommerce candidate retrieval using RetailRocket interaction data. This project is retrieval/candidate generation, not full ranking/reranking.

Completed work:

- Weighted popularity baseline.
- ID-only two-tower model.
- History-aware query tower.
- Event-weighted history pooling.
- Category-aware item tower.
- Metrics: Recall@K, HitRate@K, NDCG@K.
- Capped popularity comparison for fairer comparison with learned models.
- Reproducibility improvements through explicit random seeds.
- Mentor review documents and codebase walkthrough.

Current findings:

- Training uses filtered full train data, around 1.35M interaction rows.
- Evaluation is capped at 10k evaluable users per split for local runtime.
- Capped popularity baseline remains strongest; Test Recall@50 is around 0.0205.
- ID-only two-tower is weak because RetailRocket is sparse and many users have very few interactions.
- History-aware tower improves over ID-only.
- Event weighting and category-aware tower show some signal but are not clear wins.

Current caveats:

- Current evaluation uses train-only history to predict held-out validation/test interactions.
- It is not yet session-prefix next-item evaluation such as $[A] \rightarrow B$, $[A, B] \rightarrow C$.
- Codebase scan confirms LogQ correction, ANN/FAISS/ScaNN, retrieval demo/export embeddings, and session-prefix evaluation are not implemented yet.
- Training uses in-batch negatives with CrossEntropyLoss; explicit negative sampling is not implemented.
- `src/models/two_tower.py` should be reviewed/refactored before adding more model complexity.

4. Working method concern

I want to be transparent about my current method. I have been using AI/Codex heavily to implement fast. This is useful for speed, but I recognize that if I only click accept and continue generating more code, I may not understand the underlying concepts, metrics, configurations, and trade-offs deeply enough.

To control this, I am shifting to a gated workflow:

small scope → clear DoD → commands run → tests/reports → codebase walkthrough → limitation review → mentor feedback before expanding.

I have also created codebase walkthrough documents for both repos so I can review folder structure, key scripts, tests, commands, artifacts, and known caveats.

5. Proposed next steps

For Spark/API, I propose prioritizing production/API workflow next:

1. Add cache observability: hit_count, miss_count, cache_size, hit_rate.
2. Run load test, stress test, and performance report.
3. Compare cache vs no-cache latency.
4. Finalize API docs and add PII/privacy notes.
5. Later, generate real prediction output using LR threshold 0.20 and optionally make API data path configurable between mock data and real model predictions.

For Two-Tower, I propose not adding advanced features immediately. Possible next directions:

1. Evaluation direction: design session-prefix next-item evaluation.
2. Engineering/demo direction: export embeddings and build a simple brute-force retrieval demo.

3. Code quality direction: review/refactor two_tower.py before adding LogQ, ANN/ScaNN, or more complex training logic.

6. Questions for mentor

1. Is the current Spark/API direction aligned with the expected industry skillset: PySpark processing, prediction output, API, docs, cache/performance, and privacy awareness?
2. Is it acceptable that Bank Marketing real data has 45k rows for modeling, while a separate 500k mock prediction table is used for API serving/load-test simulation?
3. For Spark/API, should I prioritize API cache/load/stress/performance next, or continue model improvement such as GBT threshold tuning or XGBoost Spark?
4. Is LR threshold 0.20 a reasonable MVP operating point, or should model selection be redone with a separate validation set first?
5. For Two-Tower, should the next direction be session-prefix next-item evaluation, retrieval demo/export embeddings, or code refactor/review first?
6. What level of understanding is expected for an intern using AI-assisted coding: should I be able to code most parts without AI, or is it acceptable to use AI if I can explain the flow, tests, trade-offs, and limitations clearly?

7. Summary

Both projects have progressed quickly and have working artifacts, but I do not want to continue adding features without understanding. I would like to use the review meeting to confirm whether the current direction is correct, what should be prioritized next, and how deeply I should understand and be able to reproduce the implementation without AI assistance.